

Необходимо собрать openssh-server под arm, засунуть в корневую файловую систему и запустить в qemu.

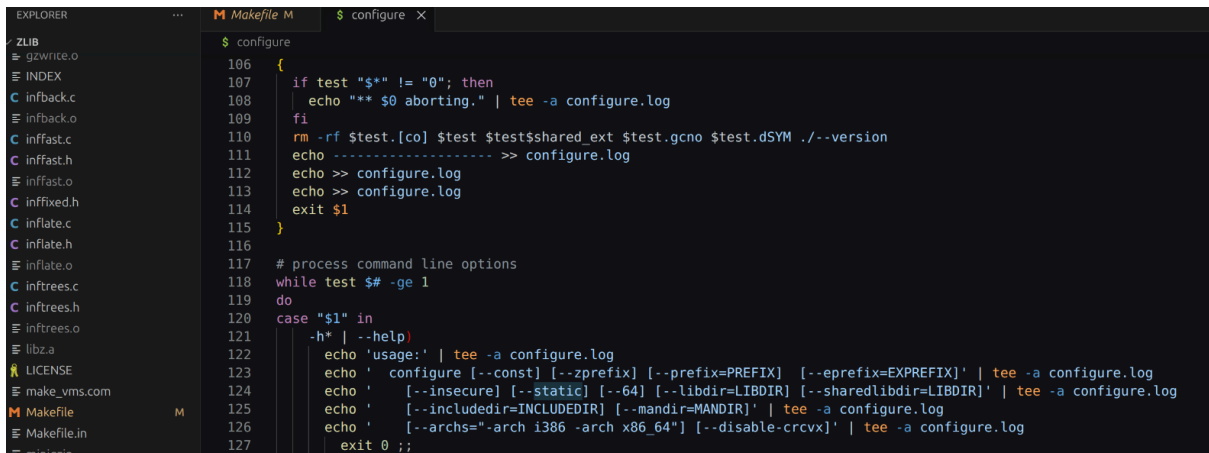
- 1) Узнал, какие библиотеки необходимы

openssh-portable / INSTALL 

Code Blame 286 lines (198 loc) · 9.88 KB

```
8      - clang: -ftrapv and -sanitize=integer require the compiler-rt runtime
9      (CC=clang LDFLAGS=--rtlib=compiler-rt ./configure)
10
11      To support Privilege Separation (which is now required) you will need
12      to create the user, group and directory used by sshd for privilege
13      separation.  See README.privsep for details.
14
15
16      The remaining items are optional.
17
18      A working installation of zlib:
19      Zlib 1.1.4 or 1.2.1.2 or greater (earlier 1.2.x versions have problems):
20      https://zlib.net/
21
22      libcrypto from one of LibreSSL, OpenSSL, AWS-LC or BoringSSL.  Building
23      without libcrypto is supported but severely restricts the available
24      ciphers and algorithms.
25      - LibreSSL (https://www.libressl.org/) 3.1.0 or greater
26      - OpenSSL (https://www.openssl.org) 1.1.1 or greater
27      - AWS-LC (https://github.com/aws/aws-lc)
28      - BoringSSL (https://github.com/google/boringssl)
29
30      libcrypto should be compiled as a position-independent library
```

- 2) Склонировал репозитории
- 3) Собираю zlib



```
$ configure
106 {
107   if test "$*" != ""; then
108     echo "*** $0 aborting." | tee -a configure.log
109   fi
110   rm -rf $test.[co] $test $test$shared_ext $test.gcno $test.dSYM ./--version
111   echo ----- >> configure.log
112   echo >> configure.log
113   echo >> configure.log
114   exit $1
115 }
116
117 # process command line options
118 while test $# -ge 1
119 do
120   case "$1" in
121     -h* | --help)
122       echo 'usage:' | tee -a configure.log
123       echo ' configure [--const] [--zprefix] [--prefix=PREFIX] [--exec-prefix=EXPPREFIX]' | tee -a configure.log
124       echo '       [--insecure] [--static] [--64] [--libdir=LIBDIR] [--sharedlibdir=LIBDIR]' | tee -a configure.log
125       echo '       [--includedir=INCLUDEDIR] [--mandir=MANDIR]' | tee -a configure.log
126       echo '       [--archs="-arch i386 -arch x86_64"] [--disable-crcv]' | tee -a configure.log
127       exit 0 ;;
```

можно определить prefix

```

grogu@osio:~/project/cross_dir/zlib$ ./configure --prefix=$(pwd)/_install --static
Checking for gcc...
Building static library libz.a version 1.3.2.1-motley with gcc.
Checking for size_t... Yes.
Checking for off64_t... Yes.
Checking for fseeko... Yes.
Checking for strerror... Yes.
Checking for unistd.h... Yes.
Checking for stdarg.h... Yes.
Checking whether to use vs[n]printf() or s[n]printf()... using vs[n]printf().
Checking for vsnprintf() in stdio.h... Yes.
Checking for return value of vsnprintf()... Yes.
Checking for attribute(visibility) support... Yes.
Checking for s390x build ... No.

```

Запуск автотулс. На выходе получился Makefile.

```

grogu@osio:~/project/cross_dir/zlib$ make CC=arm-linux-gnueabi-gcc
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -I. -c -o example.o test/example.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o adler32.o adler32.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o crc32.o crc32.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o deflate.o deflate.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o infback.o infback.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o inffast.o inffast.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o inflate.o inflate.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o inftrees.o inftrees.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o trees.o trees.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o zutil.o zutil.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o compress.o compress.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o uncompress.o uncompress.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o gzclose.o gzclose.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o gzlib.o gzlib.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o gzread.o gzread.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o gzwrite.o gzwrite.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -o example example.o -L. libz.a
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -o minigzip test/minigzip.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -o minigzip minigzip.o -L. libz.a
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -I. -D_FILE_OFFSET_BITS=64 -c -o example64.o test/example.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -o example64 example64.o -L. libz.a
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -I. -D_FILE_OFFSET_BITS=64 -c -o minigzip64.o test/minigzip.c
arm-linux-gnueabi-gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -o minigzip64 minigzip64.o -L. libz.a
grogu@osio:~/project/cross_dir/zlib$ ls

```

Переопределил компилятор в Makefile.

```

grogu@osio:~/project/cross_dir/zlib$ file gzread.o
gzread.o: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), not stripped
grogu@osio:~/project/cross_dir/zlib$ ^C

```

4) Собираю openssl

```

grogu@osio:~/project/cross_dir/openssl$ git branch
* master
grogu@osio:~/project/cross_dir/openssl$ git checkout OpenSSL_1_1_1
Updating files: 100% (21442/21442), готово.
Примечание: переключение на «OpenSSL_1_1_1».

```

Переключил ветку

Файл конфигуры у openssl написан на perl. Есть `--prefix [no-]shared`
`--cross-compile-prefix`
`--openssldir`

```
my $usage="Usage: Configure [no-<cipher> ...] [enable-<cipher> ...] [-Dxxx] [-lxxx] [-Lxxx]
[-fxxx] [-Kxxx] [no-hw-xxx|no-hw] [[no-]threads] [[no-]shared] [[no-]zlib|zlib-dynamic]
[no-asm] [no-dso] [no-egd] [sctp] [386] [--prefix=DIR] [--openssldir=OPENSSSLDIR]
[--with-xxx[=vvv]] [--config=FILE] os/compiler[:flags]\n";
```

```
--cross-compile-prefix=PREFIX
```

```
The PREFIX to include in front of commands for your
toolchain. It's likely to have to end with dash, e.g.
a-b-c- would invoke GNU compiler as a-b-c-gcc, etc.
Unfortunately cross-compiling is too case-specific to
put together one-size-fits-all instructions. You might
have to pass more flags or set up environment variables
to actually make it work. Android and iOS cases are
discussed in corresponding Configurations/15-*.conf
files. But there are cases when this option alone is
sufficient. For example to build the mingw64 target on
Linux "--cross-compile-prefix=x86_64-w64-mingw32-"
works. Naturally provided that mingw packages are
installed. Today Debian and Ubuntu users have option to
install a number of prepackaged cross-compilers along
with corresponding run-time and development packages for
"alien" hardware. To give another example
"--cross-compile-prefix=mipsel-linux-gnu-" suffices
in such case. Needless to mention that you have to
invoke ./Configure, not ./config, and pass your target
name explicitly. Also, note that --openssldir refers
to target's file system, not one you are building on.
```

```
--openssldir=DIR
```

```
Directory for OpenSSL configuration files, and also the
default certificate and key store. Defaults are:
```

```
Unix:          /usr/local/ssl
Windows:       C:\Program Files\Common Files\SSL
               or C:\Program Files (x86)\Common Files\SSL
OpenVMS:       SYS$COMMON:[OPENSSL-COMMON]
```

```
--prefix=DIR
```

```
The top of the installation directory tree. Defaults are:
```

```
Unix:          /usr/local
Windows:       C:\Program Files\OpenSSL
               or C:\Program Files (x86)\OpenSSL
OpenVMS:       SYS$COMMON:[OPENSSL-'version']
```

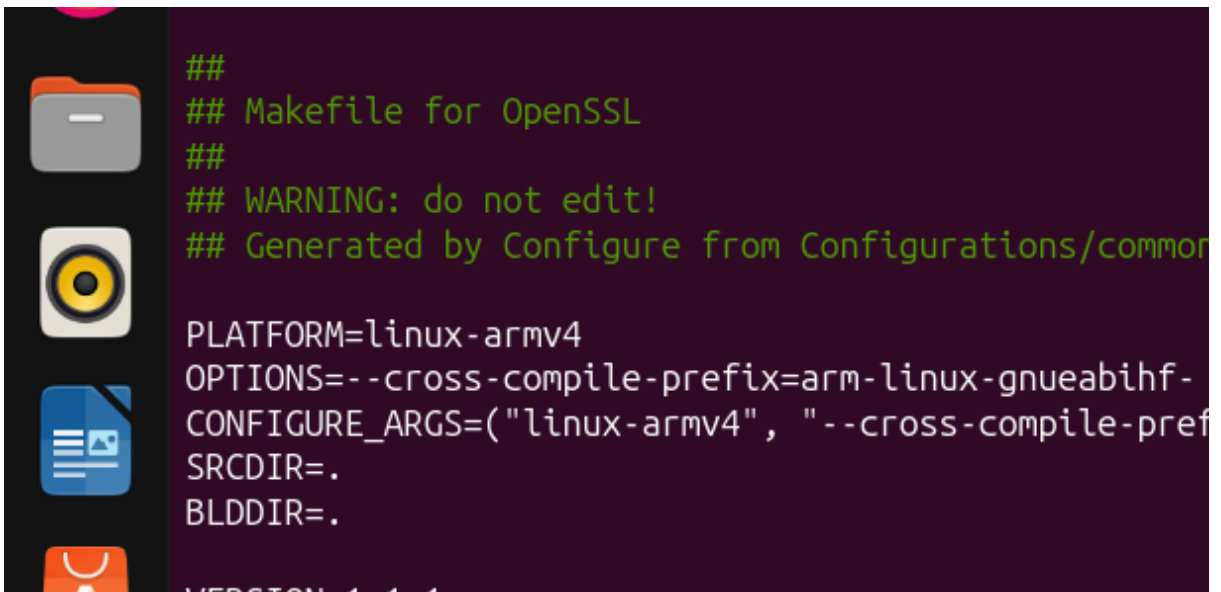
```

grogu@osio: ~/project/cross_dir/openssl$ ./configure --cross-compile-prefix=arm-linux-gnueabi --prefix=/usr
Configuring OpenSSL version 1.1.1 (0x1010100fL) for
Using os-specific seed configuration
Usage: Configure [no-<cipher> ...] [enable-<cipher> ...] [-Dxxx] [-lxxx] [-Lxxx] [-fxxx] [-Kxxx] [no-hw-xxx]
[no-egd] [sctp] [386] [--prefix=DIR] [--openssldir=OPENSSLDIR] [--with-xxx[=vvv]] [--config=FILE] os/cc
pick os/compiler from:
BS2000-OSD BSD-generic32 BSD-generic64 BSD-ia64 BSD-sparc64 BSD-sparcv8
BSD-x86 BSD-x86-elf BSD-x86_64 Cygwin Cygwin-i386 Cygwin-i486 Cygwin-i586
Cygwin-i686 Cygwin-x86 Cygwin-x86_64 DJGPP MPE/iX-gcc UEFI UWIN VC-CE VC-WIN32
VC-WIN32-ARM VC-WIN32-ONECORE VC-WIN64-ARM VC-WIN64A VC-WIN64A-ONECORE
VC-WIN64A-masm VC-WIN64I aix-cc aix-gcc aix64-cc aix64-gcc android-arm
android-arm64 android-armeabi android-mips android-mips64 android-x86
android-x86_64 android64 android64-aarch64 android64-mips64 android64-x86_64
bsdi-elf-gcc cc darwin-i386-cc darwin-ppc-cc darwin64-debug-test-64-clang
darwin64-ppc-cc darwin64-x86_64-cc dist gcc haiku-x86 haiku-x86_64
hpux-ia64-cc hpux-ia64-gcc hpux-parisc-cc hpux-parisc-gcc hpux-parisc1_1-cc
hpux-parisc1_1-gcc hpux64-ia64-cc hpux64-ia64-gcc hpux64-parisc2-cc
hpux64-parisc2-gcc hurd-x86 ios-cross ios-xcrun ios64-cross ios64-xcrun
iossimulator-xcrun iphoneos-cross irix-mips3-cc irix-mips3-gcc irix64-mips4-cc
irix64-mips4-gcc linux-aarch64 linux-alpha-gcc linux-aout linux-arm64ilp32
linux-armv4 linux-c64xplus linux-elf linux-generic32 linux-generic64
linux-ia64 linux-mips32 linux-mips64 linux-ppc linux-ppc64 linux-ppc64le
linux-sparcv8 linux-sparcv9 linux-x32 linux-x86 linux-x86-clang linux-x86_64
linux-x86_64-clang linux32-s390x linux64-mips64 linux64-s390x linux64-sparcv9
mingw mingw64 nextstep nextstep3.3 purify sco5-cc sco5-gcc solaris-sparcv7-cc
solaris-sparcv7-gcc solaris-sparcv8-cc solaris-sparcv8-gcc solaris-sparcv9-cc
solaris-sparcv9-gcc solaris-x86-gcc solaris64-sparcv9-cc solaris64-sparcv9-gcc
solaris64-x86_64-cc solaris64-x86_64-gcc tru64-alpha-cc tru64-alpha-gcc
uClinux-dist uClinux-dist64 uclinux-pro-2.0 uclinux-pro-2.1 uclinux-pro-2

```

говорит, что надо выбрать архитектуру

В Makefile указана архитектура



```

##
## Makefile for OpenSSL
##
## WARNING: do not edit!
## Generated by Configure from Configurations/common

PLATFORM=linux-armv4
OPTIONS=-cross-compile-prefix=arm-linux-gnueabi-hf-
CONFIGURE_ARGS=("linux-armv4", "-cross-compile-pref
SRCDIR=.
BLDDIR=.

```

Запустил make и make install

```

grogu@osio:~/project/cross_dir/openssl/_install/bin$ file openssl
openssl: ELF 32-bit LSB pie executable, ARM, EABI5 version 1 (SYSV), dynamically linked,
for GNU/Linux 3.2.0, not stripped
grogu@osio:~/project/cross_dir/openssl/_install/bin$ |

```

Сборка makefile

```
/configure --prefix=$(pwd)/_install --with-zlib=../zlib/_install --with-ssl-dir=../openssl/_install
--host=arm-linux-gn
ueabihf
```

Сборка ssh

make -j6

```
auth2-passwd.c      dispatch.h          m4                  regress
grogu@osio:~/project/cross_dir/openssh-portable$ file packet.o
packet.o: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), with debug_info, not
grogu@osio:~/project/cross_dir/openssh-portable$ |
```

```
uired at runtime the shared libraries from the glibc version used for linking
/usr/bin/arm-linux-gnueabi-hf-ld.bfd: ./libssh.a(misc.o): в функции «a2port»:
/home/grogu/project/cross_dir/openssh-portable/misc.c:588:(.text+0x1300): предупреждение: Using 'getservbyname' i
ries from the glibc version used for linking
grogu@osio:~/project/cross_dir/openssh-portable$ file sshd
sshd: ELF 32-bit LSB executable, ARM, EABI5 version 1 (GNU/Linux), statically linked, BuildID[sha1]=b39e475deff3c
t stripped
grogu@osio:~/project/cross_dir/openssh-portable$ |
```

```
grogu@osio:~/project/cross_dir/openssh-portable$ ls -la ssh sshd scp sftp ssh-keygen ssh-keyscan
-rwxrwxr-x 1 grogu grogu 760476 Jun 18 02:50 scp
-rwxrwxr-x 1 grogu grogu 792576 Jun 18 02:50 sftp
-rwxrwxr-x 1 grogu grogu 5322788 Jun 18 02:50 ssh
-rwxrwxr-x 1 grogu grogu 4046772 Jun 18 02:50 ssh-keygen
-rwxrwxr-x 1 grogu grogu 4058972 Jun 18 02:50 ssh-keyscan
-rwxrwxr-x 1 grogu grogu 4147608 Jun 18 02:50 sshd
grogu@osio:~/project/cross_dir/openssh-portable$ |
```

создание паролей

```
grogu@osio:~/project/busybox/busybox-1.35.0/_install$ cat > etc/passwd << 'EOF'
root:x:0:0:root:/root:/bin/sh
sshd:x:100:65534:sshd:/var/empty:/bin/false
EOF
grogu@osio:~/project/busybox/busybox-1.35.0/_install$ |
```

создание архива

```
find . | cpio -o -H newc | gzip > ~/project/initramfs_with_ssh.cpio.gz
```

Запуск эмулятора

```
grogu@osio:~/project/firmware$ ls
```

```
1.c init initramfs.cpio.gz script.sh vexpress-v2p-ca9.dtb zImage
```

```
grogu@osio:~/project/firmware$ QEMU_AUDIO_DRV=none qemu-system-arm -M
vexpress-a9 -kernel zImage -initrd initramfs.cpio.gz -dtb vexpress-v2p-ca9.dtb -append
"console=ttyAMA0 rdinit=bin/ash" -nographic
```

```

Couldn't open /dev/null: No such file or directory
/ # sshd
Couldn't open /dev/null: No such file or directory
/ # ls
bin      etc      linuxrc  sbin     var
dev      init     root     usr
/ # cd [ 17.136294] amba 1000f000.wdt: deferred probe pending: (reason unknown)
[ 17.136438] amba 100e0000.memory-controller: deferred probe pending: (reason unknown)
[ 17.136464] amba 100e1000.memory-controller: deferred probe pending: (reason unknown)
[ 17.136485] amba 100e5000.watchdog: deferred probe pending: (reason unknown)
sbin
/sbin # ls
acpid      hwclock      lsmod        run-init
adjtimex   ifconfig     makedevs     runlevel
arp        ifdown       mdev         setconsole
blkid      ifenslave    mkdosfs      slattach
blockdev   ifup         mke2fs       sshd
bootchartd init         mkfs.ext2    start-stop-daemon
depmod     insmod       mkfs.minix   sulogin
devmem     ip          mkfs.vfat    swapoff
fb splash   ipaddr       mkswap       swapon
fdisk      iplink      modinfo      switch_root
findfs     ipneigh     modprobe     sysctl
freeramdisk iproute     nameif       syslogd
fsck       iprule      pivot_root   tuncctl
fsck.minix iptunnel    poweroff     udhcpc
fstrim     klogd       raidautorun  uevent
getty      loadkmap    reboot       vconfig
halt       logread     rmmmod       watchdog
hdparm     losetup     route        zcip
/sbin # sshd
Couldn't open /dev/null: No such file or directory
/sbin # |

```